2 Water Jug Problem

Aim

Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.

Theory

The 2 Water Jug Problem is a common puzzle in AI that shows how state-space search works which is a key idea in problem solving. The puzzle has two jugs with different sizes like a 4-liter jug and a 3-liter jug. The goal is to measure out a specific amount of water in one jug by using only allowed actions and an unlimited water supply.

To solve this puzzle we treat it as a search problem:
State Representation: Each step of the puzzle is called a state. A state is written as an ordered pair ((x, y)), where (x) is the amount of water in the first jug and (y) is the amount in the second jug.

The Initial State is ((0, 0)), meaning both jugs start empty.
The Goal State is any state that meets the requirement, such as ((2, y)) if the goal is to get 2 liters in the first jug.

Operators (Moves): These are the allowed actions that change one state into another.
Common actions include:

- Filling a jug completely from the water source.
- Emptying a jug onto the ground.
- Pouring water from one jug into the other until one jug is full or the other is empty.

A solution is found using a search method like Breadth-First Search (BFS) which explores all possible states. The algorithm finds a valid sequence of actions that leads from the starting

state to the goal state. The jug sizes and pouring rules limit the possible moves, which makes the problem clear and manageable.

Algorithm / Methodology

1. INITIALIZATION:
   - Set current contents of Jug X (xValue) = 0.
   - Set current contents of Jug Y (yValue) = 0.
   - Set capacities (xCap, yCap) and the goal (target).

2. LOOP (Continue until target reached):

   A. TERMINATION CHECK:
      - If xValue equals target OR yValue equals target:
        - Print success message.
        - Exit loop.

   B. USER INPUT:
      - Display list of choices (Fill, Empty, Pour).
      - Prompt user for index (1-6).

   C. STATE TRANSITION (Conditionals):
      - CHOICE 1 (Fill X): xValue = xCap.
      - CHOICE 2 (Fill Y): yValue = yCap.
      - CHOICE 3 (Empty X): xValue = 0.
      - CHOICE 4 (Empty Y): yValue = 0.
      - CHOICE 5 (Pour X -> Y):
        - IF total (xValue + yValue) > yCap:
          - xValue = xValue - (yCap - yValue)
          - yValue = yCap
        - ELSE:
          - yValue = xValue + yValue
          - xValue = 0

- CHOICE 6 (Pour Y -> X):
  - IF total (yValue + xValue) > xCap:
    - yValue = yValue - (xCap - xValue)
    - xValue = xCap
  - ELSE:
    - xValue = xValue + yValue
    - yValue = 0

D. OUTPUT:
- Display updated (xValue, yValue).

Dataset Description

Attributes

The dataset consists of the following attributes that define the environment and the current state:

Attribute	Type	Description
xCap	Integer	The maximum capacity of Jug X.
yCap	Integer	The maximum capacity of Jug Y.
target	Integer	The desired volume of water to be measured.
xValue	Integer	The current volume of water in Jug X (0 to xCap).

yValue	Integer	The current volume of water in Jug Y (0 to yCap).
choice	Integer	The action selected by the user (1-6).

Size

The size of the state space is finite and is determined by the formula:

Size = (xCap + 1) * (yCap + 1)

For the input twoJugs(3, 5, 4):

Total possible states: (3 + 1) * (5 + 1) = 24 possible coordinate states.
Reachability: While there are 24 possible mathematical states, only a subset of these are reachable through the legal moves (Fill, Empty, Pour) defined in your algorithm.

Implementation

```
# mani func
def main():
    x = 0
    y = 0

    twoJugs(3, 5, 4)

# take capacity of Jugs and target
def twoJugs(xCap, yCap, target):
```

```
    # set initial jug values to 0
    xValue = 0
    yValue = 0

    choices = ["1. Fill X",
               "2. Fill Y",
               "3. Empty X",
               "4. Empty Y",
               "5. Pour X->Y",
               "6. Pour Y->X"]

    while(1):

        if (xValue == target and yValue == 0):

            print(f"\nGoal Reached\nX contains {target}ltr
water")
            break

        elif (yValue == target and xValue == 0):
            print(f"\nGoal Reached\nY contains {target}ltr
water")
            break

        else:
            print(f"\n{choices}\n")
            choice = int(input("Enter index of choice: ")) - 1
            print(f"Chose: {choices[choice]}")
```

```
if (choice == 0):
    xValue = xCap

elif (choice == 1):
    yValue = yCap

elif (choice == 2):
    xValue = 0

elif (choice == 3):
    yValue = 0

elif (choice == 4):
    if (yValue + xValue > yCap):
        xValue -= (yCap - yValue)
        yValue = yCap

    else:
        yValue = xValue + yValue
        xValue = 0
else:
    if (yValue + xValue > xCap):
        yValue -= (xCap - xValue)
        xValue = xCap

    else:
        xValue = xValue + yValue
        yValue = 0
```

```
        print(f"X: {xValue}, Y: {yValue}")

main()
```

Output

```
['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  2
Chose: 2. Fill Y
X: 0, Y: 5

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  6
Chose: 6. Pour Y->X
X: 3, Y: 2

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  3
Chose: 3. Empty X
X: 0, Y: 2

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  6
Chose: 6. Pour Y->X
X: 2, Y: 0

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  2
Chose: 2. Fill Y
X: 2, Y: 5

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  6
Chose: 6. Pour Y->X
X: 3, Y: 4
```

```
['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  6
Chose: 6. Pour Y->X
X: 2, Y: 0

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  2
Chose: 2. Fill Y
X: 2, Y: 5

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  6
Chose: 6. Pour Y->X
X: 3, Y: 4

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  1
Chose: 1. Fill X
X: 3, Y: 4

['1. Fill X', '2. Fill Y', '3. Empty X', '4. Empty Y', '5. Pour X->Y', '6. Pour Y->X']

Enter index of choice:  3
Chose: 3. Empty X
X: 0, Y: 4

Goal Reached
Y contains 4ltr water
```

Result Analysis

I observed that the program successfully tracks the volume of water in each jug through a series of state changes triggered by user input. By executing the predefined rules for filling, emptying, and pouring, I was able to reach the target of 4L in Jug Y by following a specific sequence of moves. The output confirms that the program correctly handles the overflow logic during transfers, ensuring that neither jug exceeds its maximum capacity while maintaining the total volume of water during the process.

Conclusion

Through this experiment, I learned how to model the Two Jugs puzzle as a state-space search problem using conditional programming. I gained a deeper understanding of how to implement state transitions and how to define termination conditions based on specific goal states. I learned that this puzzle is based on a specific math rule: the goal amount must be divisible by the greatest common divisor (GCD) of the two jug sizes. If it isn't, the problem is

impossible to solve. This experiment showed me how to take a logic puzzle and turn it into a working interactive program.

Missionaries and Cannibals Problem

Aim

Write a program to implement Missionaries and Cannibals problem.
Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory

The Missionaries and Cannibals problem is a classic river-crossing puzzle used in Artificial Intelligence to demonstrate state-space search techniques. The problem involves three missionaries and three cannibals who must cross a river using a boat that holds a maximum of two people. The primary constraint is the safety rule: at no point on either bank can the cannibals outnumber the missionaries, as the missionaries would be eaten. Additionally, the boat cannot move across the river without at least one person on board.

To solve this computationally, the problem is modeled as a directed graph where each node represents a state and each edge represents a valid move. A state is defined by a tuple (C,M,B), representing the number of cannibals and missionaries on the starting bank and the current location of the boat. For this specific implementation, I defined the boat as being on the left bank (B=0) or the right bank (B=1).

I utilized the Breadth-First Search (BFS) algorithm to navigate this state space. BFS is particularly suited for this problem because it explores the graph layer-by-layer, ensuring that

the first time the goal state (0,0,1) is reached, it is via the shortest path. During the search, a "visited" set is maintained to record explored states. This is critical because the moves are reversible (e.g., moving two people across and then immediately back); without tracking visited states, the algorithm would enter an infinite loop. The complexity of the problem lies in the narrow "safe" path through the state space, as many mathematically possible moves lead to invalid states where the safety constraint is violated.

Algorithm / Methodology

Initialize the State: Define a state as (C, M, B), where C is cannibals on the left, M is missionaries on the left, and B is the boat position (0 for left, 1 for right)

1. Define Initial and Goal States:
    - Initial State: (3, 3, 0) (All on the left bank)
    - Goal State: (0, 0, 1) (All on the right bank)
2. Setup Data Structures:
    - Create a Queue and add the initial state to it along with an empty path
    - Create a Visited list and add the initial state to prevent cycles
3. Start the Search Loop (Repeat while the queue is not empty):
    - Pop the first node (C, M, B) and its current path from the queue (FIFO)
    - Check Goal: If the current state matches the Goal State, return the path
4. Generate Possible Moves:
    - Iterate through all possible boat loads (dc, dm): (1,0), (2,0), (0,1), (0,2), (1,1)
5. Calculate New State:
    - If B=0 (Left to Right): New State = (C - dc, M - dm, 1)
    - If B=1 (Right to Left): New State = (C + dc, M + dm, 0)
6. Validate and Transition:
    - Boundary Check: Ensure C and M are between 0 and 3
    - Constraint Check: Ensure missionaries are not outnumbered on either bank:
        - On Left: M >= C (if M > 0)
        - On Right: (3-M) >= (3-C) (if (3-M) > 0)
    - Duplicate Check: If the state is valid and not in Visited:
        - Add New State to Visited

        - Append the New State and the updated path to the Queue
7. Terminate: If the queue becomes empty and no goal is found

Dataset Description

Attributes

The state of the system is given by 3 tuple (C, M, B):

1. C (Cannibals): {0, 1, 2, 3} being possible count of cannibals on the left bank
2. M (Missionaries): {0, 1, 2, 3} being possible count of cannibals on the left bank
3. B (Boat): {0, 1} being the location of boat. 0 means left side and 1 means right side.

Size

1. Total Possible States: 4×4×2=32 total combinations
2. Valid States: After applying the safety constraints (missionaries ≥ cannibals on both banks), the reachable state space consists of only 20 valid states.
3. Transitions: There are 5 possible operators (moves) per state, creating a directed graph structure.

Implementation

```
# main func
def main():
    # Boat: 0 = Left, 1 = Right
    initial_state = (3, 3, 0)
    goal_state = (0, 0, 1)

    # calling bfs algo
    bfs(initial_state, goal_state)
```

```
# checks if the state is valid i.e. the missionaries are safe
def is_valid(c, m):
    if c < 0 or c > 3 or m < 0 or m > 3:
        return False

    # missionaries cannot be outnumbered by cannibals on left
bank
    if m > 0 and m < c:
        return False

    # same on right bank
    if (3 - m) > 0 and (3 - m) < (3 - c):
        return False

    return True

# bfs to find possible states
def bfs(start, goal):
    queue = [[start, []]]
    visited = [start]

    while queue:
        current_node = queue.pop(0)
        (c, m, boat) = current_node[0]
        path = current_node[1]

        if (c, m, boat) == goal:
            print("Goal Reached")
```

```
            full_path = path + [(c, m, boat)]

            for i, step in enumerate(full_path):
                b_pos = "Left" if step[2] == 0 else "Right"
                print(f"Step {i}\n C:{step[0]}, M:{step[1]},
Boat:{b_pos}")

            return

        # possible moves
        moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

        for dc, dm in moves:

            if boat == 0:
                new_state = (c - dc, m - dm, 1)

            else:
                new_state = (c + dc, m + dm, 0)

            # validate state and check if already visited
            if is_valid(new_state[0], new_state[1]) and
new_state not in visited:
                visited.append(new_state)
                queue.append([new_state, path + [(c, m,
boat)]])

    print("No solution")
```

```
main()
```

Output

```
Goal Reached
Step 0
 C:3, M:3, Boat:Left
Step 1
 C:1, M:3, Boat:Right
Step 2
 C:2, M:3, Boat:Left
Step 3
 C:0, M:3, Boat:Right
Step 4
 C:1, M:3, Boat:Left
Step 5
 C:1, M:1, Boat:Right
Step 6
 C:2, M:2, Boat:Left
Step 7
 C:2, M:0, Boat:Right
Step 8
 C:3, M:0, Boat:Left
Step 9
 C:1, M:0, Boat:Right
Step 10
 C:2, M:0, Boat:Left
Step 11
 C:0, M:0, Boat:Right
```

Result Analysis

The BFS algorithm successfully identifies the optimal 11-step solution to transport all individuals to the right bank. The output demonstrates that the algorithm avoids invalid states where missionaries are outnumbered, such as (2, 1, 0), and ignores redundant cycles by utilizing a visited list.

By exploring the state space layer-by-layer, BFS guarantees the shortest path, proving that a sequence of moves exists that satisfies all safety constraints. The transition from the initial state (3, 3, 0) to the goal state (0, 0, 1) is achieved without any missionary casualties, validating the logic of the state-space search.

Conclusion

I modeled the Missionaries and Cannibals problem as a state-space search and implemented BFS to guarantee finding the shortest 11-move solution. I found that maintaining a visited list was mandatory to prevent the boat from entering infinite loops between identical bank states. This experiment taught me how to automate complex safety constraints ensuring missionaries are never outnumbered—to solve logic puzzles more reliably than manual human reasoning.